

Section 8.4

The input: embeddings for token and position

Flow dễ nói, dễ hiểu, có ví dụ minh họa

Nguyễn Thái Sơn

Mục tiêu của phần này

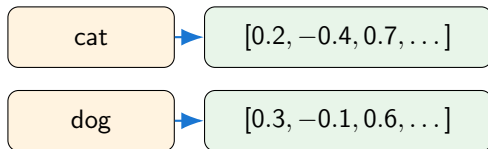
- Hiểu **word embedding** là gì.
- Hiểu vì sao cần thêm **positional embedding**.
- Thấy được cách tạo đầu vào cuối cùng của Transformer:

$$X = X_{\text{word}} + X_{\text{position}}$$

Ý tưởng trình bày: đi từ *khái niệm* → *bảng embedding* → *one-hot* → *vị trí*.

Câu hỏi mở đầu: máy tính hiểu từ bằng cách nào?

- Máy tính không hiểu trực tiếp chữ như con người.
- Vì vậy, mỗi từ cần được đổi thành **vector số**.
- Vector này được gọi là **word embedding**.



Ví dụ trực quan:

“cat” $\rightarrow [0.2, -0.4, 0.7, \dots]$

“dog” $\rightarrow [0.3, -0.1, 0.6, \dots]$

Word Embedding là gì?

- Mỗi token được ánh xạ thành một vector trong không gian d chiều.
- Ta lưu toàn bộ các vector này trong một **ma trận embedding**:

$$E \in \mathbb{R}^{|V| \times d}$$

- $|V|$: kích thước từ vựng.
- d : số chiều embedding.
- **Mỗi hàng của E** là embedding của một từ trong vocabulary.

Cách hiểu đơn giản: E giống như một **bảng tra cứu vector** của các từ.

Ví dụ minh họa: ma trận embedding là một bảng tra cứu

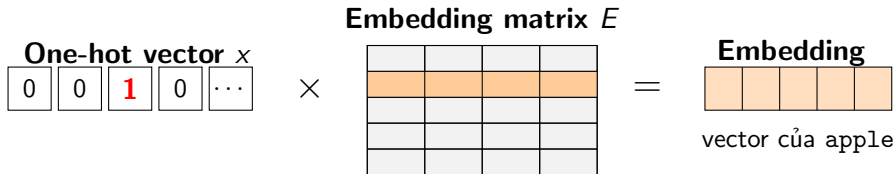
Embedding matrix E

cat					
dog					
apple					
book					
⋮					

Mỗi hàng là **một vector embedding**

Muốn lấy embedding của một từ, ta chỉ cần chọn đúng hàng của từ đó trong bảng E .

Lấy embedding bằng phép nhân ma trận



$$x = [0 \ 0 \ 1 \ 0 \ \dots] \Rightarrow xE = e_{\text{apple}}$$

One-hot vector giúp chọn đúng một hàng trong ma trận E .

Trước khi lấy embedding: one-hot là gì?

- Để máy tính biết phải chọn **hàng nào** trong bảng E , ta cần biểu diễn từ bằng **one-hot vector**.
- One-hot vector là vector mà:
 - tất cả phần tử đều bằng 0
 - chỉ có **một** vị trí bằng 1

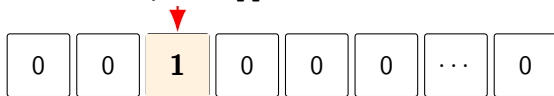
Ví dụ: nếu từ apple có index là 3, thì

$$x = [0 \ 0 \ 1 \ 0 \ 0 \ 0 \ \dots \ 0]$$

Số 1 chỉ đúng vị trí của từ trong vocabulary.

Ví dụ hình ảnh: one-hot vector

Ví dụ: từ apple có index = 3



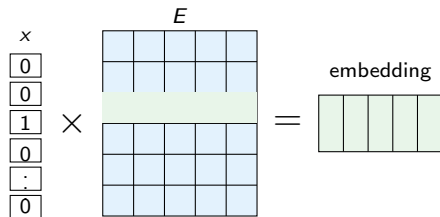
One-hot vector: chỉ một vị trí bằng 1

Từ one-hot đến embedding

- One-hot vector sẽ được nhân với ma trận embedding E .
- Kết quả là chọn ra đúng hàng tương ứng trong E .

$$xE = \text{embedding của token}$$

Ý nghĩa: phép nhân này thực chất chỉ là một cách toán học để **lấy đúng hàng cần tra**.



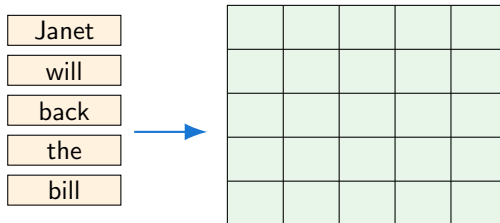
Biểu diễn cả câu

- Với một câu gồm N token, ta làm tương tự cho từng token.
- Kết quả là ma trận word embedding:

$$X_{\text{word}} \in \mathbb{R}^{N \times d}$$

$$X_{\text{word}} \in \mathbb{R}^{N \times d}$$

Câu: Janet will back the bill



Mỗi hàng là embedding của một token trong câu.

Vấn đề còn lại: mô hình chưa biết thứ tự từ

- Word embedding cho mô hình biết **từ là gì**.
- Nhưng chưa cho biết **từ đứng ở đâu trong câu**.
- Đây là vấn đề lớn vì thứ tự từ ảnh hưởng trực tiếp đến ý nghĩa.

Ví dụ:

“I love you” $\neq$ “You love I”

I love you

You love I

Cùng từ, khác thứ tự

Giải pháp: Positional Embedding

- Transformer không đọc tuần tự như RNN.
- Vì vậy, ta phải thêm thông tin vị trí cho từng token.
- Mỗi vị trí i có một vector riêng p_i .

$$x_i = e_i + p_i$$

- e_i : word embedding của token thứ i
- p_i : positional embedding của vị trí thứ i

Word embedding nói “từ là gì”, positional embedding nói “từ đứng ở đâu”.

Ví dụ hình ảnh: cộng word embedding và positional embedding

Word		Position		Kết quả
e_1	+	p_1	=	$x_1 = e_1 + p_1$
e_2	+	p_2	=	$x_2 = e_2 + p_2$
e_3	+	p_3	=	$x_3 = e_3 + p_3$
e_4	+	p_4	=	$x_4 = e_4 + p_4$
e_5	+	p_5	=	$x_5 = e_5 + p_5$

Positional Embedding được tạo như thế nào?

- Ta tạo một ma trận vị trí:

$$P \in \mathbb{R}^{N \times d}$$

- N : độ dài tối đa của chuỗi
- d : số chiều embedding
- Mỗi hàng thứ i là vector p_i

Ý tưởng

- Mỗi vị trí có một vector riêng
- Ví dụ:
 - vị trí 1 $\rightarrow p_1$
 - vị trí 2 $\rightarrow p_2$
 - vị trí 3 $\rightarrow p_3$

p_i được học như thế nào?

- Positional embedding được tạo theo cách:
 - 1 Khởi tạo ngẫu nhiên ma trận P
 - 2 Trong quá trình training:
 - Model tự điều chỉnh các giá trị của P

Kết luận

- p_i là vector học được (learnable)
- Không phụ thuộc vào từ, chỉ phụ thuộc vào vị trí

$$x_i = e_i + p_i$$

Một cách khác: Sin/Cos Positional Encoding

- Ngoài cách học trực tiếp, ta có thể dùng công thức:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

- Không cần học (không phải learnable)
- Tổng quát tốt cho chuỗi dài hơn

Đầu vào cuối cùng của Transformer

- Sau khi cộng hai thành phần, ta thu được ma trận đầu vào cuối cùng:

$$X = X_{\text{word}} + X_{\text{position}}$$

$$X \in \mathbb{R}^{N \times d}$$

Ý nghĩa

- X_{word} : mang thông tin ngữ nghĩa.
- X_{position} : mang thông tin thứ tự.
- X : là đầu vào được đưa vào các Transformer blocks.